INF221 — Lecture 2

Today's topics

1. Proof by induction
2. Insertion sort—analysing performance
3. Designing algorithms

Proof by induction

This type of proof is used often when proving the correctness of algorithms. Inductive proofs can only be used for propositions that can be enumerated, and that have a definite "smallest" case: We have a proposition for $n = 1$, $n = 2$, $n = 3$, $\ldots$, which we can call $P(1)$, $P(2)$, $\ldots$, $P(n)$, $\ldots$.

We then prove the proposition first for the *base case*, usually $P(1)$. We then *assume* that $P(2), P(3), \ldots, P(n)$ all hold and prove that $P(n+1)$ also holds under this assumption. Thus we have proven the proposition for all n.

Example

Proposition

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

This is the general form $P(n)$ of the proposition.

Proof

Base case (initialization)

The base case is

$$P(1): \qquad \sum_{k=1}^{1} k = \frac{1(1+1)}{2} \tag{1}$$

$$\iff \qquad 1 = \frac{2}{2} = 1 \tag{2}$$

We have thus proven the base case.

Induction step (maintenance)

We have proven that $P(1)$ holds and assume that $P(2), P(3), \ldots, P(n)$ all hold. We especially assume that for any n we choose

$$P(n): \qquad \sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

holds. We now need to show that the proposition holds for $n+1$, i.e., that

$$P(n+1): \qquad \sum_{k=1}^{n+1} k = \frac{(n+1)(n+2)}{2}$$

is true.

We have

$$\begin{aligned}
\sum_{k=1}^{n+1} k &= \left[\sum_{k=1}^{n} k\right] + (n+1) && (3)\\
&= \frac{n(n+1)}{2} + (n+1) \qquad \text{used } P(n) && (4)\\
&= \frac{n(n+1)}{2} + \frac{2(n+1)}{2} && (5)\\
&= \frac{n(n+1) + 2(n+1)}{2} && (6)\\
&= \frac{(n+2)(n+1)}{2} \qquad \text{qed.} && (7)
\end{aligned}$$

Insertion Sort—Analyzing performance

- Compendium, part 2
- Performance is essential if programs are to be useful
- We need to assess performance theoretically (paper and pencil) and empirically (benchmarks)
- Key question: how does **running time** depend on **input size**
- Basic assumptions in algorithm analysis:
  - All operations require approximately the same time
  - The number of operations is essential

Analysis of Insertion Sort

Line	Statement	Cost	Number of executions
2	`for j = 2 to A.length`	c_2	n
3	`  key = A[j]`	c_3	$n-1$
4	`  i = j - 1`	c_4	$n-1$
5	`  while i > 0 and A[i] > key`	c_5	$\sum_{j=2}^{n} t_j$
6	`  A[i + 1] = A[i]`	c_6	$\sum_{j=2}^{n} (t_j - 1)$
7	`  i = i - 1`	c_7	$\sum_{j=2}^{n} (t_j - 1)$
8	`  A[i + 1] = key`	c_8	$n-1$

where

- $n =$ `A.length`
- t_j is number of times the `while`-loop test is performed for given `j`

The total runtime is then

$$
\begin{aligned}
T(n) &= c_2 n + c_3(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) \\
&= c_2 n + (c_3 + c_4 + c_8)(n-1) + (c_5 + c_6 + c_7) \sum_{j=2}^{n} t_j - (c_6 + c_7)(n-1) \\
&= c_2 n + (c_3 + c_4 + c_8 - c_6 - c_7)(n-1) + (c_5 + c_6 + c_7) \sum_{j=2}^{n} t_j \\
&= (c_5 + c_6 + c_7) \sum_{j=2}^{n} t_j + (c_2 + c_3 + c_4 + c_8 - c_6 - c_7)n - (c_3 + c_4 + c_8 - c \\
&= A \sum_{j=2}^{n} t_j + Bn + C
\end{aligned}
$$

- What is t_j?

Best case

- We never enter the while loop: $t_j = 1$ for all j
- True if data is already sorted
- Then we have

$$
T(n) = A \sum_{j=2}^{n} 1 + Bn + C = A(n-1) + Bn + C = (A+B)n + (C-A)
$$

- Running time is **linear** in n
- Considering only the term of highest order in n, we can express this as

$$T(n) = \Theta(n)$$

Worst case

- We have to run through the loop until `i==0` for each j
- True if data are in reverse order
- Then $t_j = j$
- Then we have

$$T(n) = A\sum_{j=2}^{n} j + Bn + C = A\left(\frac{n(n+1)}{2} - 1\right) + Bn + C = \frac{A}{2}n^2 + \left(\frac{A}{2}\right.$$

- Running time is **quadratic** in n
- We can express this as

$$T(n) = \Theta(n^2)$$

Average case

- What does "average" mean?
- Assume here: need to run half-way through while-loop
- Then $t_j = j/2$
- Quadratic running-time dependence as for worst case, but with $A/4$ instead of $A/2$
- In this case, we also have

$$T(n) = \Theta(n^2)$$

For insertion sort, the average case essentially behaves like the worst case.

Benchmarks of Insertion Sort

- Let us test the theory by performing simulation experiments

In [ ]:
```
def insertion_sort(A):
    for j in range(1, len(A)):
        key = A[j]
        i = j - 1
        while i >= 0 and A[i] > key:
            A[i + 1] = A[i]
            i = i - 1
        A[i + 1] = key
```

Best case

In [ ]:
```
%timeit insertion_sort(list(range(100)))
```

```
5.12 µs ± 5.95 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```

In [ ]:
```
%timeit insertion_sort(list(range(1000)))
```

```
63 µs ± 138 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

In [ ]:
```
%timeit insertion_sort(list(range(10000)))
```

```
684 µs ± 11.6 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

In [ ]:
```
%timeit insertion_sort(list(range(100000)))
```

```
6.89 ms ± 133 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

In [ ]:
```
%timeit insertion_sort(list(range(1000000)))
```

```
72.1 ms ± 287 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

Worst case

In [ ]:
```
%timeit insertion_sort(list(reversed(range(100))))
```

```
203 µs ± 813 ns per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

In [ ]:
```
%timeit insertion_sort(list(reversed(range(1000))))
```

```
22.3 ms ± 96.8 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

In [ ]:
```
%timeit insertion_sort(list(reversed(range(10000))))
```

```
2.32 s ± 29.2 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

In [ ]:
```
import sys
sys.version
```

```
'3.14.2 (main, Dec  5 2025, 16:49:16) [Clang 17.0.0 (clang-1700.6.3.2)]'
```

Average case

In [ ]:
```
from random import random
```

In [ ]:
```
%timeit insertion_sort([random() for _ in range(100)])
```

```
106 µs ± 62 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

In [ ]:
```
%timeit insertion_sort([random() for _ in range(1000)])
```

```
11 ms ± 28.2 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Results for different Python versions

- Obtained with `ins_sort_bench.py` script
- Apple MacBook Pro with M3 CPU
- Timing execution of `insertion_sort(list(reversed(range(1000)))`

Python version	Runtime	Comment
3.9.25	31.4 ms ± 0.2 ms	Best of three runs (33.5±0.2, 32.0±0.3)
3.10.19	30.0 ms ± 0.2 ms	
3.11.14	19.5 ms ± 0.2 ms	FasterPython project results

for the significant difference is unclear. | | 3.12.12 | 21.4 ms ± 0.3 ms | Slight performance regression | | 3.13.11 | 24.0 ms ± 0.1 ms | Significant performance regression | | 3.14.2 | 21.9 ms ± 0.1 ms | Some performance improvement |

... compared to last year's data

from the same computer

Python version	Runtime	Comment
3.9.21	29.4 ms ± 0.4 ms	
3.10.16	28.3 ms ± 0.3 ms	
3.11.11	11.9 ms ± 0.0 ms	FasterPython project results
3.12.8	13.5 ms ± 0.1 ms	Slight performance regression
3.13.1	18.2 ms ± 0.2 ms	Significant performance regression

- What has happened?
- Most likely due to different Python binaries

Comparing Python 3.14 installed from different sources

Source	Runtime
conda-forge	13 ms
homebrew	22 ms
python.org	18 ms

- Algorithm analysis is about which complexity class (e.g., linear vs quadratic) an algorithm belongs to
- Crucial to determine the size of problems we can handle
- Implementation details, including hardware, system software and software build options can have a significant effect on actual runtime, but do not influence the complexity class an algorithm belongs to.

Designing algorithms

In [1]:
```python
def insertion_sort_p(A):
    for j in range(1, len(A)):
        print(A)
        key = A[j]
        i = j - 1
        while i >= 0 and A[i] > key:
            A[i + 1] = A[i]
            i = i - 1
        A[i + 1] = key
    print(A)
```

In [2]:
```python
insertion_sort_p([5, 4, 3, 2, 1])
```

```
[5, 4, 3, 2, 1]
[4, 5, 3, 2, 1]
[3, 4, 5, 2, 1]
[2, 3, 4, 5, 1]
[1, 2, 3, 4, 5]
```

- We have considered the **insertion sort** algorithm that uses an *incremental* approach for sorting
  - starting with a sorted subarray `A[1..j-1]`
  - inserting a single element `A[j]` into proper place
  - after algorithm finished: yield sorted subarray `A[1..j]`
- **Insertion sort** algorithm is effecient for sorting small number of elements
- In the worst-case scenario, **insertion sort** algorithm is slow since it is computationally expensive

Problem size development for insertion sort

Consider sorting data originally in reverse order

5	4	3	2	1
4	5	3	2	1
3	4	5	2	1
2	3	4	5	1
1	2	3	4	5

For each round through the outer loop, one more element is sorted. Or, looking the other way around, one fewer element remains unsorted. Thus, we *reduce problem size by one for each round through the outer loop*.

Can we do better?

Divide-and-conquer approach

- The **divide-and-conquer** algorithm design can perform the same sorting task (for worst case scenario) in far less time than **insertion sort**
- Many useful algorithms are *recursive*
- Recursion in computer science is a method where the solution to a problem depends on solutions to smaller instances of the same problem
- Recursion is one of the central ideas of computer science

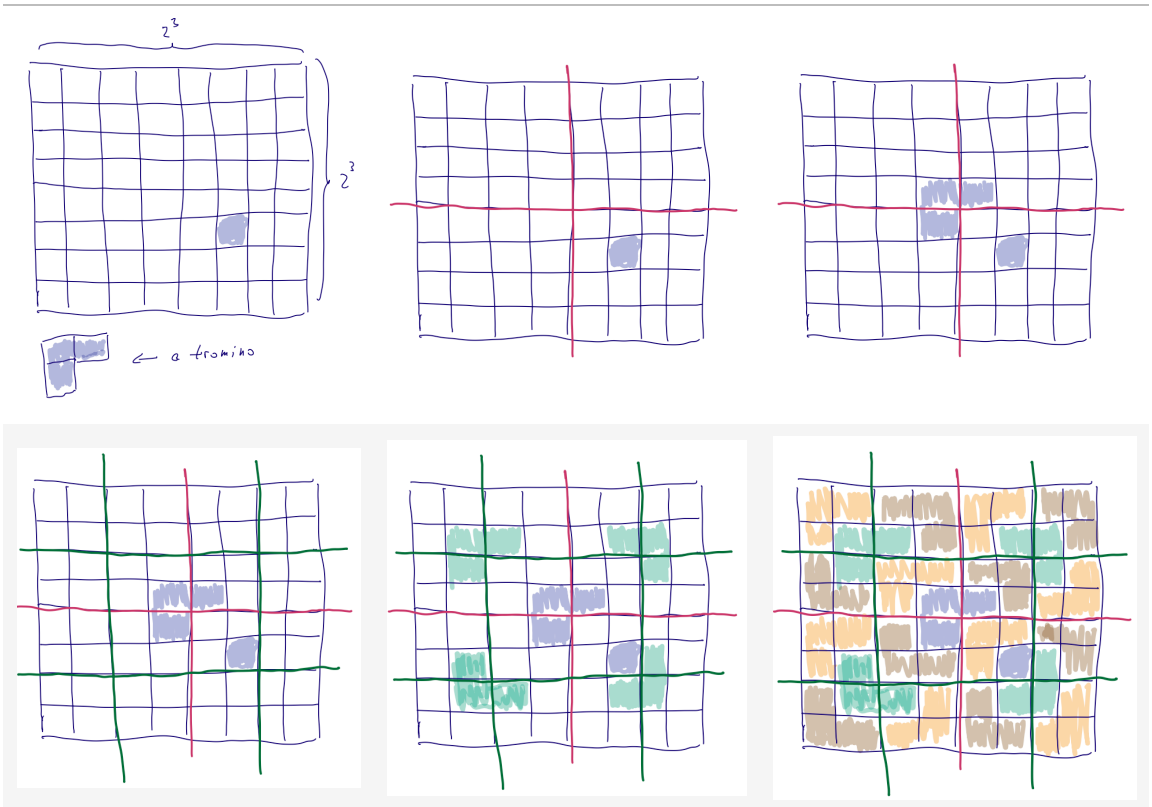
Divide-and-conquer paradigm involves *three* steps at each level of *recursion*.

1. **Divide** the problem into a number of subproblems
2. **Conquer** the subproblems by solving them *recursively*. If the subproblem sizes are small enough, however, just solve the subproblems in a straightforward manner
3. **Combine** the solutions to the subproblem into the solution for the original problem.

Tromino — an example of a problem best solved recursively

- We are given a field with $2^n \times 2^n$ squares, for $n \in \mathbb{N}_0$.
- Exactly one of these squares is filled.
- Fill the remaining squares by placing L-shaped trominos into the field!

See also Wikipedia article on Trominos.



Tromino algorithm

1. Divide the problem evenly into four subproblems. Only one subproblem will contain a filled square.
2. Place a single tromino so that it fills exactly one square in each of the three empty subproblems ("place it around the corner of the subproblem with the filled square").
3. Now all four subproblems contain exactly one filled square, i.e., we have four problems of exactly the same type as the original problem, but with size $2^{n-1} \times 2^{n-1}$.
4. Repeat 1–3 for the subproblems, stopping at 2×2 subproblems.

Comments

- One can visually "see" that the algorithm will work. We will not attempt a formal proof here.
- It is a typical and visual example of breaking larger problems into smaller problems *of exactly the same type* and solving those.
- Because the subproblems are of exactly the same type, we can solve them with exactly the same algorithm.
- This is the power of *recursion* (from Latin "re-" for "again" and "currere" for "to run").
- Recursion is one of the most powerful and important concepts in computer science. We will see many examples during the course. See also the Wikipedia article on recursion in computer science).

Challenge

A little reflection over the Tromino problem above suggests the following mathematical theorem:

> For every $n \in \mathbb{N}_0$, there exists an $m \in \mathbb{N}_0$ such that $4^n = 3m + 1$.

Can you argue informally why this should hold, and can you give a mathematical proof by induction?

This theorem describes a relation between integers. This leads to quite different mathematics than the calculus of real numbers. Such equations are a part of *discrete mathematics* and are quite important for computer science and cryptography.

Divide-and conquer paradigm in the ***merge sort*** algorithm

See Compendium, part 3

1. **Divide:** Divide the *n*-element sequence to be sorted into two subsequences of $n/2$

2. **Conquer:** Sort the two subsequences recursively using *merge sort*
3. **Combine:** Merge the two sorted subsequences to produce the sorted answer

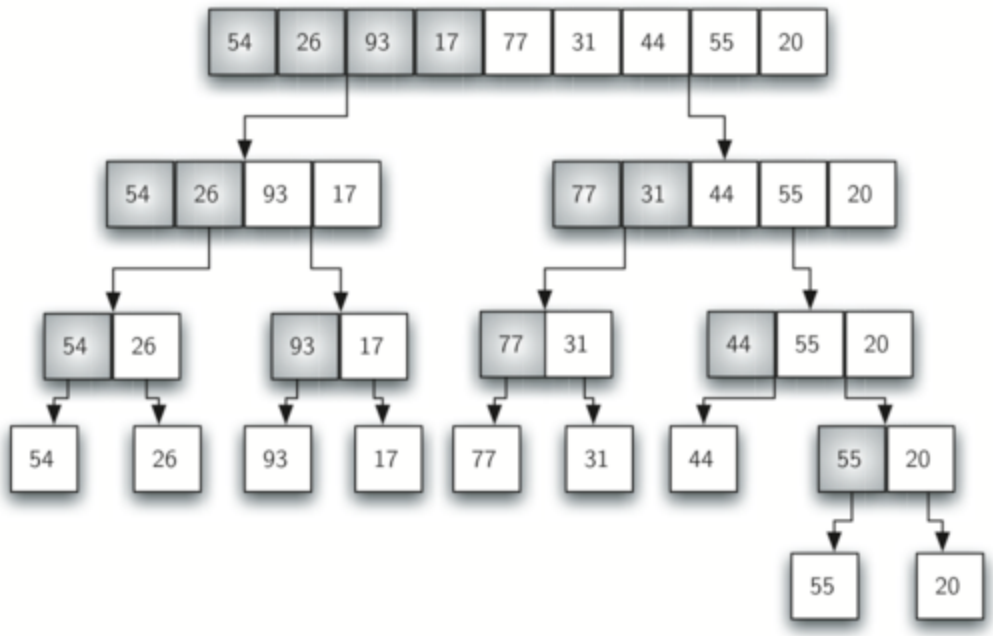
Note that the recursion "bottoms out" when sequence to be sorted has length 1.

Key operation in **merge sort**: the **merge** procedure

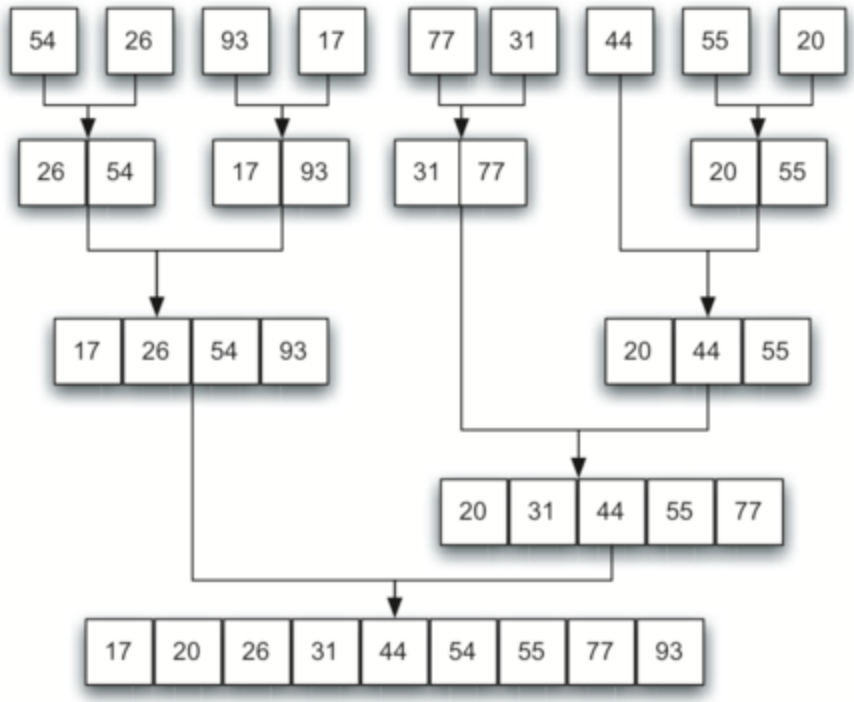
Graphical illustration of merge-sort procedure

Figures from http://www.interactivepython.org (now defunct).

Dividing process



Merging process



Pseudocode for merge procedure

Input:

- Array A with length n
- Indices $p \leq q < r \leq n$
- Subarrays $A[p \ldots q]$ and $A[q+1 \ldots r]$ are each sorted

Output:

- Array A with the following properties
  - Subarray $A[p \ldots r]$ is sorted
  - Subarrays $A[1 \ldots p-1]$ and $A[r+1 \ldots n]$ unchanged

```
MERGE(A, p, q, r)
n1 = q - p + 1
n2 = r - q
let L[1..n1 + 1] and R[1..n2 + 1] be new arrays
for i = 1 to n1
    L[i] = A[p + i - 1]
for j = 1 to n2
    R[j] = A[q + j]
L[n1 + 1] = ∞
R[n2 + 1] = ∞
i = 1
j = 1
for k = p to r
```

```
13:     if L[i] ≤ R[j]
14:         A[k] = L[i]
15:         i = i + 1
16:     else
17:         A[k] = R[j]
18:         j = j + 1
```

The infinite values at `L[n1+1]` and `R[n2+1]` are *sentinels* which simplify the code on lines 12–18: we do not need to check if we run out of elements in `L` or `R` inside the for loop.

Python implementation of merge procedure

This Python implementation is designed to be as close to the pseudocode as possible. It is **not** elegant Python code.

In [3]:
```python
def merge(A, p, q, r):
    n1 = q - p + 1
    n2 = r - q

    L = [0] * n1
    R = [0] * n2

    for i in list(range(n1)):
        L[i] = A[p + i - 1]

    for j in list(range(n2)):
        R[j] = A[q + j]

    L.append(float('inf'))
    R.append(float('inf'))

    i = 1 - 1     # Subtract 1 to adjust to Python indexing
    j = 1 - 1     # Subtract 1 to adjust to Python indexing

    for k in list(range(p - 1, r)):     # Subtract 1 from q to adjust to Pyt
        if L[i] <= R[j]:
            A[k] = L[i]
            i = i + 1
        else:
            A[k] = R[j]
            j = j + 1


a = [78, 77, 76, 75, 74, 73, 72, 71, 2, 4, 5, 7, 1, 2, 3, 6, -81, -82, -83]
p_idx = 9
q_idx = 12
r_idx = 16

merge(a, p_idx, q_idx, r_idx)
print('Sorted output', a)
```

```
Sorted output [78, 77, 76, 75, 74, 73, 72, 71, 1, 2, 2, 3, 4, 5, 6, 7, -81,
-82, -83]
```

Execute in Online Python Tutor

Correctness of `Merge(A, p, q, r)`

Invariant

At the start of each iteration of the **for** loop of lines 12-18

1. subarray `A[p..k - 1]` contains the `k - p` smallest elements of `L[1..n1 + 1]` and `R[1..n2 + 1]` in sorted order
2. `L[i]` and `R[j]` are the smallest elements of their arrays that have not been copied back into `A`

Proof

Initialization

- Prior to first loop: `k = p`, which means that subarray `A[p..k - 1]` is empty
- Moreover, the empty subarray `A[p..k - 1]` contains the `k - p = 0` smallest elements of `L` and `R`
- Since `i = j = 1` and `L` and `R` are sorted by prerequisite, both `L[i]` and `R[j]` are the smallest elements of their arrays that have not been copied back into `A`

Maintenance

- If `L[i] ≤ R[j]`, then `L[i]` is smallest element not yet copied back into `A`
  - Since `A[p..k - 1]` contains the `k - p` smallest elements, the subarray `A[p..k]` will contain the `k - p + 1` smallest elements after line 14 copies `L[i]` into `A[k]`
  - Incrementing `k` and `i` in line 15 reestablishes loop invariant for next iteration
- If instead `L[i] > R[j]`, then lines 16-17 perform appropriate action to maintain loop invariant

Termination

- At termination, `k = r + 1`
- At this point, `A[p..k - 1]` turns into `A[p..r]` and contains `k - p = r - p + 1` smallest elements of `L[1..n1 + 1]` and `R[1..n2 + 1]`, in sorted order
- Arrays `L` and `R`
  - together contain `n1 + n2 + 2 = r - p + 3` elements
  - of those elements all but the two largest elements have been copied back to `A`
  - the two largest elements are sentinels which were not part of `A[p..r]` and are discarded

- Therefore, `A[p..r]` is now sorted.

Running times for merge procedure

The procedure runs in $\Theta(n)$ time:

- n is defined as $n = r - p + 1$
- Each of lines 1–3 and 8–11 takes constant time
- **for** loops of lines 4–7 take $\Theta(n_1 + n_2) = \Theta(n)$
- $n + 1$ iterations of **for** loops of lines 12–18, each of which take constant time

Merge-sort procedure

- Use *merge* procedure as a subroutine inside merge-sort algorithm
- `merge_sort(A, p, r)` sorts elements in the subarray `A[p..r]`
- If $p \geqslant r$, the subarray has **at most** *one* element and is therefore already sorted
- Otherwise: partition `A[p..r]` into two subarrays `A[p..q]` and `A[q+1..r]` with $q = \lfloor (p + r)/2 \rfloor$

Pseudocode for merge-sort procedure

```
MERGE-SORT(A, p, r)
1:  if p < r
2:      q = ⌊(p + r)/2⌋
3:      MERGE-SORT(A, p, q)
4:      MERGE-SORT(A, q + 1, r)
5:      MERGE(A, p, q, r)
```

To sort a complete array, call `MERGE-SORT(A, 1, A.length)`.

Floor and Ceiling

- *Floor*: $\lfloor x \rfloor$ is the largest integer $\leq x$
- *Ceiling*: $\lceil x \rceil$ is the smallest integer $\geq x$
- For $x \in \mathbb{Z}$ we have $\lfloor x \rfloor = \lceil x \rceil$

In-class exercise

1. What are $\lfloor \frac{3}{2} \rfloor$, $\lceil \frac{3}{2} \rceil$, $\lfloor -\frac{3}{2} \rfloor$, $\lceil -\frac{3}{2} \rceil$?
2. Show that $\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$ for all $n \in \mathbb{N}_0$!
3. Show that choosing $q = \lfloor (p + r)/2 \rfloor$ yields subarrays $A[p \ldots q + 1]$ and $A[q \ldots r]$ with lengths $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$ respectively

---

Analysing divide-and-conquer algorithms

- When algorithm contains recursive call to itself:
  - running time can be often described by ***recurrence equation*** or ***recurrence***
  - ***recurrence*** describes overall running time on a problem of size n
  - does so in terms of the running time on smaller inputs
- Can use mathematical tools to:
  - solve the recurrence
  - provide bounds on the performance of the algorithm
- See Compendium Part 3 and 5

Recurrence for running time of divide-and-conquer algorithm

- $T(n)$ is the running time on a problem of size n
- If problem size small enough:
  - that is, $n \leq c$ for some constant c:
  - straightforward solution takes constant time $\Theta(1)$
- Otherwise: division of problem yields a subproblems
  - each subproblem is of size $1/b$ of the original problem
  - for *merge-sort* both a and b are 2
  - note: for many *divide-and-conquer* algorithms $a \neq b$
  - time needed to solve one subproblem is $T(n/b)$
  - time needed to divide original problem into subproblems is $D(n)$
  - time needed to combine solutions to the subproblems into the solution to the original problem is $C(n)$

The above can be written in the following way

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

We will discuss methods for solving recurrences later.

A few notes on recurrence-based analysis

- Note that the pseudocode for *merge sort* works correctly even if number of elements is *not even*
- Recurrence-based analysis is simplified:
  - it assumes that original problem size is a power of 2 (i.e., that number of elements is even) despite the fact that the problem has an odd number of elements
  - this assumption implies that the size of two subsequences is **exactly** $n/2$

- one can show that this assumption does not affect order of growth of solution of reccurence

Setting up recurrence for $T(n)$ for worst-case running time of merge sort on n numbers

- If merge sort is applied on just **one element** it takes constant time
- If $n > 1$:
  - **Divide**
    - computes the middle of the subarray
    - this computation takes constant time (not dependent on size of subarray)
    - therefore $D(n) = \Theta(1)$
  - **Conquer**
    - recursively solves two subproblems
    - each subproblem is of size $n/2$
    - in sum all subproblems contribute $2T(n/2)$ to the running time
  - **Combine**
    - Already established above that **merge** procedure on an n-element subarray takes time $\Theta(n)$
    - therefore $C(n) = \Theta(n)$

For **merge sort** with $a = 2$, $b = 2$, $D(n) = \Theta(1)$, and $C(n) = \Theta(n)$

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

becomes

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

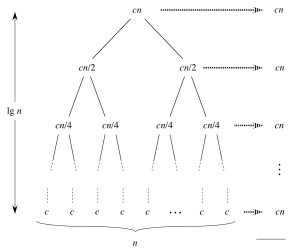
Note that since the sum of $D(n) + C(n) = \Theta(1) + \Theta(n)$ is a linear function of n the result is $\Theta(n)$.

If we use a constant c to represent the time to solve problems of size 1, as well as the time per array element of the divide and combine steps, we can rewrite the recurrence the following way:

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$

Recursion tree

Before we look at formal methods to solve the recurrence above, we can obtain a solution by looking at the recursion tree.



Source: *CLSR Fig 2.5*

- Assumes that n is exact power of 2
- At the root (top level): the cost is cn
- At a specific level i below top:
  - level has 2^i nodes, each contributing a cost of $c(n/2^i)$
  - the total cost across all nodes at this level is $2^i c(n/2^i) = cn$
  - Example: at level $i = 2$ below top
    - there are $2^2 = 4$ nodes
    - the cost for one node is $c(n/4)$
    - the total cost across all nodes is $4 \times c(n/4) = cn$
- Total number of levels is $\lg n + 1$, where n is the number of leaves (corresponding to input size)
- To compute the total cost represented by recurrence the costs at all levels are added up
  - recurrence tree has $\lg n + 1$ levels
  - the cost at each level is cn
  - the total cost: $cn(\lg n + 1) = cn \lg n + cn$
  - Ignoring low-order term and constant gives $\Theta(n \lg n)$